

LAPORAN TUGAS AKHIR
ALGORITMA DAN STRUKTUR DATA



DOSEN PENGAMPU:

Taufik Ridwan, M.T.

DISUSUN OLEH:

Kaila Rachmania Nareshwari	(2510631250035)
Reyhan Sulaiman	(2510631250044)
Shafira Ramadhani	(2510631250020)

PROGRAM STUDI SISTEM INFORMASI
FAKULTAS ILMU KOMPUTER
UNIVERSITAS SINGAPERBANGSA KARAWANG
2026

DAFTAR ISI

BAB I	2
PENDAHULUAN	2
1.1 Latar Belakang	2
1.2 Rumusan Masalah	2
1.3 Tujuan Penelitian	3
1.4 Manfaat Penelitian	3
BAB II.....	4
ANALISIS FITUR & KEBUTUHAN	4
2.1 Analisis Kebutuhan Sistem	4
2.2 Fitur Sistem.....	4
CRUD (Create, Read, Update, Delete)	4
Searching.....	4
Sorting.....	4
Pembelian Tiket	5
2.3 Struktur Data yang Digunakan.....	5
BAB III	6
IMPLEMENTASI	6
3.1 Struktur Data (Array of Records)	6
3.2 Implementasi Fungsi Program	7
BAB IV	9
ANALISIS KOMPLEKSITAS	9
4.1 Analisis Kompleksitas Waktu (<i>Time Complexity</i>).....	9
4.2 Penjelasan Kompleksitas Waktu	9
□ Operasi Berkompleksitas Konstan $O(1)$	9
□ Operasi Berkompleksitas Linear $O(n)$	10
□ Operasi Berkompleksitas Kuadratik $O(n^2)$	10
□ Analisis Fungsi <i>cariID()</i>	11
4.3 Analisis Kompleksitas Ruang (<i>Space Complexity</i>).....	11
BAB V	12
PENGUJIAN.....	12
BAB VI	16
KESIMPULAN.....	16

BAB I

PENDAHULUAN

1.1 Latar Belakang

Sebagai mahasiswa, kami memilih tema sistem bioskop karena sistem ini cukup familiar dan mudah dipahami dalam kehidupan sehari-hari, tetapi tetap memiliki alur program yang kompleks untuk dijadikan project Algoritma dan Struktur Data. Melalui tema bioskop, kami dapat menerapkan berbagai materi yang telah dipelajari selama perkuliahan ke dalam satu program yang saling terhubung.

Kami memilih project ini karena dalam sistem bioskop terdapat banyak proses yang sesuai dengan materi pembelajaran, seperti pengelolaan data film, pencarian data, pengurutan data, pemesanan tiket, hingga pemilihan kursi. Fitur-fitur tersebut memungkinkan kami untuk mengimplementasikan konsep CRUD, searching, sorting, array object, serta beberapa algoritma seperti Linear Search, Binary Search, Bubble Sort, dan Selection Sort.

Selain untuk memenuhi tugas UAS, project ini juga bertujuan untuk melatih kemampuan logika pemrograman, kerja sama tim, serta pemahaman kami dalam membuat alur sistem yang terstruktur. Dengan membuat simulasi sistem bioskop berbasis Java console, kami dapat lebih memahami bagaimana algoritma dan struktur data diterapkan dalam kasus nyata secara sederhana.

1.2 Rumusan Masalah

1. Bagaimana cara membuat sistem pemesanan tiket bioskop sederhana menggunakan bahasa pemrograman Java?
2. Bagaimana penerapan konsep CRUD (Create, Read, Update, Delete) pada sistem bioskop?
3. Bagaimana implementasi algoritma searching dan sorting pada pengelolaan data film?
4. Bagaimana cara membuat sistem pemilihan kursi dan pembelian tiket pada program bioskop berbasis console?

5. Bagaimana penerapan struktur data array object dalam penyimpanan data film bioskop?

1.3 Tujuan Penelitian

1. Membuat sistem pemesanan tiket bioskop sederhana menggunakan bahasa pemrograman Java.
2. Mengimplementasikan konsep CRUD pada pengelolaan data film bioskop.
3. Mengimplementasikan algoritma searching dan sorting dalam proses pencarian dan pengurutan data.
4. Membuat fitur pembelian tiket serta pemilihan kursi bioskop secara digital.
5. Menerapkan struktur data array object sebagai media penyimpanan data pada program.

1.4 Manfaat Penelitian

1. Membantu memahami penerapan Algoritma dan Struktur Data dalam project nyata.
2. Melatih kemampuan logika pemrograman dan penyusunan alur sistem.
3. Menambah pengalaman dalam membuat program menggunakan bahasa Java.
4. Menjadi contoh implementasi materi searching, sorting, CRUD, dan array object.
5. Membantu proses pembelajaran agar lebih mudah dipahami melalui simulasi sistem nyata.
6. Menjadi dasar pengembangan sistem bioskop yang lebih modern.
7. Dapat dikembangkan lebih lanjut menjadi sistem berbasis GUI maupun database.

BAB II

ANALISIS FITUR & KEBUTUHAN

2.1 Analisis Kebutuhan Sistem

Pada project ASD Cinema System UAS, sistem dirancang untuk membantu pengelolaan data film dan proses pemesanan tiket bioskop secara sederhana berbasis Java console. Sistem ini membutuhkan beberapa fitur utama agar program dapat berjalan sesuai kebutuhan pengguna.

Kebutuhan sistem pada program ini meliputi pengelolaan data film, pencarian data, pengurutan data, hingga proses pembelian tiket dan pemilihan kursi. Selain itu, sistem juga menggunakan struktur data array object untuk menyimpan data film serta algoritma searching dan sorting untuk mempermudah pengolahan data.

2.2 Fitur Sistem

CRUD (Create, Read, Update, Delete)

Fitur CRUD digunakan untuk mengelola data film pada sistem bioskop.

Create : menambahkan data film baru

Read : menampilkan seluruh data film

Update : mengubah data film berdasarkan ID

Delete : menghapus data menggunakan metode soft delete

Searching

Fitur searching digunakan untuk mencari data film dengan lebih mudah.

- **Cari Nama Film** menggunakan Linear Search
- **Cari Kategori Film** menggunakan Linear Search
- **Cari ID Film** menggunakan Binary Search

Sorting

Fitur sorting digunakan untuk mengurutkan data film.

- **Sort ID** menggunakan Bubble Sort (ascending)
- **Sort Nama Film** menggunakan Selection Sort (A-Z)
- **Sort Kursi Terbanyak** menggunakan Bubble Sort (descending)

Pembelian Tiket

Fitur ini digunakan untuk melakukan pemesanan tiket bioskop.

Pengguna dapat:

- Memilih hari tayang
- Memilih film
- Memilih jam tayang
- Memilih kursi bioskop
- Melihat total pembayaran
- Mendapatkan struk pembelian

Sistem kursi dibuat menggunakan array 2 dimensi untuk menampilkan posisi kursi bioskop.

Keterangan:

[] = kursi tersedia

[X] = kursi sudah dipesan

2.3 Struktur Data yang Digunakan

Tiket[] data	Array object untuk menyimpan seluruh data film bioskop
String[] jam	Array untuk menyimpan jadwal jam tayang film
String[][] seat	Array 2 dimensi untuk menyimpan posisi kursi bioskop
Class Tiket	Class object yang berisi atribut data film seperti ID, nama film, kategori, kursi, jam, dan status

BAB III

IMPLEMENTASI

3.1 Struktur Data (Array of Records)

Pada program sistem pemesanan tiket bioskop ini, struktur data utama yang digunakan adalah **Array of Records** yang diimplementasikan menggunakan class Tiket dan array objek statis.

Class Tiket berfungsi sebagai record atau representasi satu data film beserta atribut-atribut yang dimiliki. Setiap objek Tiket menyimpan informasi berupa ID film, nama film, kategori, jumlah kursi tersedia, harga tiket, hari tayang, status tiket, jadwal tayang, serta denah tempat duduk bioskop.

```
class Tiket {  
  
    int id, kursi, harga;  
    String film, kategori, hari, status;  
    String[] jam;  
    String[][] seat = new String[5][5];  
}
```

Pada class tersebut digunakan beberapa jenis struktur data, yaitu:

- Variabel primitif (int)
- String
- Array satu dimensi (String[] jam)
- Array dua dimensi (String[][] seat)

Array dua dimensi digunakan untuk merepresentasikan kursi bioskop berukuran 5×5 .

Penyimpanan seluruh objek tiket dilakukan menggunakan array statis:

```
static Tiket[] data = new Tiket[100];
```

Array tersebut memiliki kapasitas maksimal sebanyak 100 data film dan berfungsi sebagai media penyimpanan utama selama program berjalan (*runtime*).

3.2 Implementasi Fungsi Program

Program dibangun menggunakan konsep modular programming, di mana setiap fitur dipisahkan ke dalam fungsi-fungsi tertentu agar kode lebih terstruktur dan mudah dipahami.

1. Fungsi *generateData()*

Fungsi ini digunakan sebagai proses *seeding data* awal. Sistem secara otomatis membuat sekitar 30 data film lengkap dengan variasi jadwal tayang serta simulasi kondisi kursi yang sudah terisi. Tujuan utama fungsi ini adalah mempermudah proses pengujian program tanpa harus memasukkan data secara manual satu per satu.

2. *tambahData()* (**Create**)

Fungsi ini digunakan untuk menambahkan data film baru ke dalam array. Program menerima input pengguna berupa informasi film, kemudian membuat objek Tiket baru dan menyimpannya pada indeks array berikutnya menggunakan variabel penghitung (counter).

3. *tampilData()* (**Read**)

Fungsi ini bertugas menampilkan seluruh data film yang masih aktif dalam bentuk tabel pada konsol. Tampilan data diformat menggunakan perintah `System.out.printf()` agar lebih rapi dan mudah dibaca.

4. *editData()* (**Update**)

Fungsi ini melakukan pencarian data berdasarkan ID film menggunakan metode pencarian sekuensial (*Linear Search*). Jika data ditemukan, pengguna diberikan akses untuk mengubah informasi tertentu seperti judul film, kategori, dan hari tayang.

5. *hapusData()* (**Delete**)

Pada fungsi ini diterapkan konsep *Soft Delete*. Data film tidak benar-benar dihapus dari array karena dapat menyebabkan perubahan indeks data lainnya. Sebagai gantinya, atribut status diubah menjadi "Dihapus" sehingga data dianggap tidak aktif.

6. *cariNama()* dan *cariKategori()* (**Searching**)

Kedua fungsi ini digunakan untuk mencari data berdasarkan nama film atau kategori. Proses pencarian dilakukan dengan menelusuri seluruh isi array dan mencocokkan input pengguna menggunakan metode *case-insensitive* agar pencarian lebih fleksibel.

7. *sortID()*, *sortNama()*, dan *sortKursiTerbanyak()* (**Sorting**)

Fungsi-fungsi ini digunakan untuk melakukan pengurutan data film berdasarkan beberapa kriteria seperti ID, nama film, dan jumlah kursi tersisa. Seluruh proses sorting dibuat secara manual tanpa menggunakan library bawaan Java.

8. *cariID()* (**Binary Search**)

Fungsi ini digunakan untuk mencari data berdasarkan ID menggunakan algoritma *Binary Search*. Sebelum proses pencarian dilakukan, data terlebih dahulu diurutkan berdasarkan ID agar memenuhi syarat penggunaan algoritma tersebut.

9. *beliTiket()* dan *tampilKursi()* (**Fitur Utama**)

Fungsi ini merupakan inti dari sistem pemesanan tiket bioskop. Pengguna dapat memilih film, jadwal tayang, serta posisi kursi yang diinginkan. Sistem akan memetakan input kursi seperti "B3" ke dalam indeks array dua dimensi `seat[][]` menggunakan operasi aritmetika karakter.

Jika kursi masih tersedia dengan tanda "[]", maka sistem akan mengubah status kursi menjadi "[X]", mengurangi jumlah kursi tersisa, kemudian mencetak struk pembelian tiket.

BAB IV

ANALISIS KOMPLEKSITAS

4.1 Analisis Kompleksitas Waktu (*Time Complexity*)

Analisis kompleksitas waktu dilakukan untuk mengetahui tingkat efisiensi algoritma yang digunakan pada setiap fungsi program. Pengukuran dilakukan menggunakan notasi Big-O dengan n sebagai jumlah data film yang tersimpan di dalam array.

Fungsi / Fitur	Algoritma yang Digunakan	Best Case	Worst Case	Average Case
tambahData()	Direct Insertion	$O(1)$	$O(1)$	$O(1)$
tampilData()	Sequential Traversal	$O(n)$	$O(n)$	$O(n)$
editData()	Linear Search	$O(1)$	$O(n)$	$O(n)$
hapusData()	Linear Search	$O(1)$	$O(n)$	$O(n)$
cariNama()	Linear Search (Full Scan)	$O(n)$	$O(n)$	$O(n)$
cariKategori()	Linear Search (Full Scan)	$O(n)$	$O(n)$	$O(n)$
sortID()	Bubble Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
sortNama()	Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
sortKursiTerbanyak()	Bubble Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
cariID()	Bubble Sort + Binary Search	$O(n^2)$	$O(n^2)$	$O(n^2)$
beliTiket()	Linear Search + Array 2D	$O(1)$	$O(n)$	$O(n)$

4.2 Penjelasan Kompleksitas Waktu

- **Operasi Berkompleksitas Konstan $O(1)$**

Fungsi *tambahData()* memiliki kompleksitas konstan karena proses penambahan data dilakukan langsung pada indeks terakhir array tanpa perulangan yang dipengaruhi jumlah data.

```
data[n++] = new Tiket(  
id, film, kategori, 25, jam, hari);
```

Proses tersebut hanya membutuhkan satu kali operasi penyimpanan sehingga kompleksitas waktunya adalah $O(1)$.

- **Operasi Berkompleksitas Linear $O(n)$**

Fungsi *tampilData()*, *cariNama()*, dan *cariKategori()* menggunakan metode pencarian dan penelusuran sekuensial (*Linear Search*).

```
for (Tiket t : data)
```

Pada fungsi *cariNama()* dan *cariKategori()*, program tetap melakukan pemindaian seluruh data meskipun data ditemukan karena kemungkinan terdapat lebih dari satu film dengan kategori atau nama yang sama. Oleh karena itu, kompleksitas pada Best Case, Worst Case, dan Average Case tetap bernilai $O(n)$.

Fungsi *beliTiket()* juga didominasi oleh pencarian linear ketika menampilkan film berdasarkan hari dan mencari ID film yang dipilih pengguna.

Sedangkan validasi kursi menggunakan array dua dimensi berukuran tetap 5×5 sehingga dianggap konstan: Karena ukuran matriks kursi tidak berubah mengikuti jumlah data film.

- **Operasi Berkompleksitas Kuadratik $O(n^2)$**

- a. Bubble Sort (*sortID()* dan *sortKursiTerbanyak()*)

Fungsi ini menggunakan algoritma Bubble Sort dengan dua perulangan bersarang (*nested loop*).

```
for (int i = 0; i < n - 1; i++) {  
    for (int j = 0; j < n - i - 1; j++) {
```

Karena program tidak menggunakan mekanisme optimasi seperti *flagging*, maka seluruh proses perbandingan tetap dijalankan sehingga kompleksitas waktunya adalah $O(n^2)$.

- b. Selection Sort (*sortNama()*)

Fungsi *sortNama()* menggunakan algoritma Selection Sort untuk mencari elemen terkecil pada setiap iterasi.

```
if (data[j].film.compareTo(  
    data[min].film) < 0)
```

Algoritma tetap melakukan pemindaian seluruh sisa elemen array sehingga kompleksitasnya konsisten berada pada $O(n^2)$.

- **Analisis Fungsi *cariID()***

Secara teori, Binary Search memiliki kompleksitas waktu sebesar:

Namun pada program ini, fungsi *cariID()* terlebih dahulu memanggil fungsi *sortID()* sebelum melakukan pencarian.

```
sortID();
```

Karena proses sorting menggunakan Bubble Sort dengan kompleksitas $O(n^2)$, maka total kompleksitas fungsi menjadi:

Berdasarkan aturan dominasi pada notasi Big-O, kompleksitas akhir fungsi *cariID()* adalah: $O(n^2)$

4.3 Analisis Kompleksitas Ruang (*Space Complexity*)

Analisis kompleksitas ruang dilakukan untuk mengetahui penggunaan memori program selama proses eksekusi.

1. Penyimpanan Data Utama

Program menggunakan array statis sebagai media penyimpanan utama:

```
static Tiket[] data = new Tiket[100];
```

Array tersebut memiliki kapasitas tetap sebanyak 100 elemen. Jika dianalisis berdasarkan jumlah data aktif yang digunakan, maka kompleksitas ruang penyimpanan utama bernilai: $O(n)$

2. Kompleksitas Ruang pada Algoritma Sorting

Seluruh algoritma sorting pada program menggunakan metode *In-Place Sorting*, yaitu proses pengurutan dilakukan langsung pada array utama tanpa membuat array tambahan.

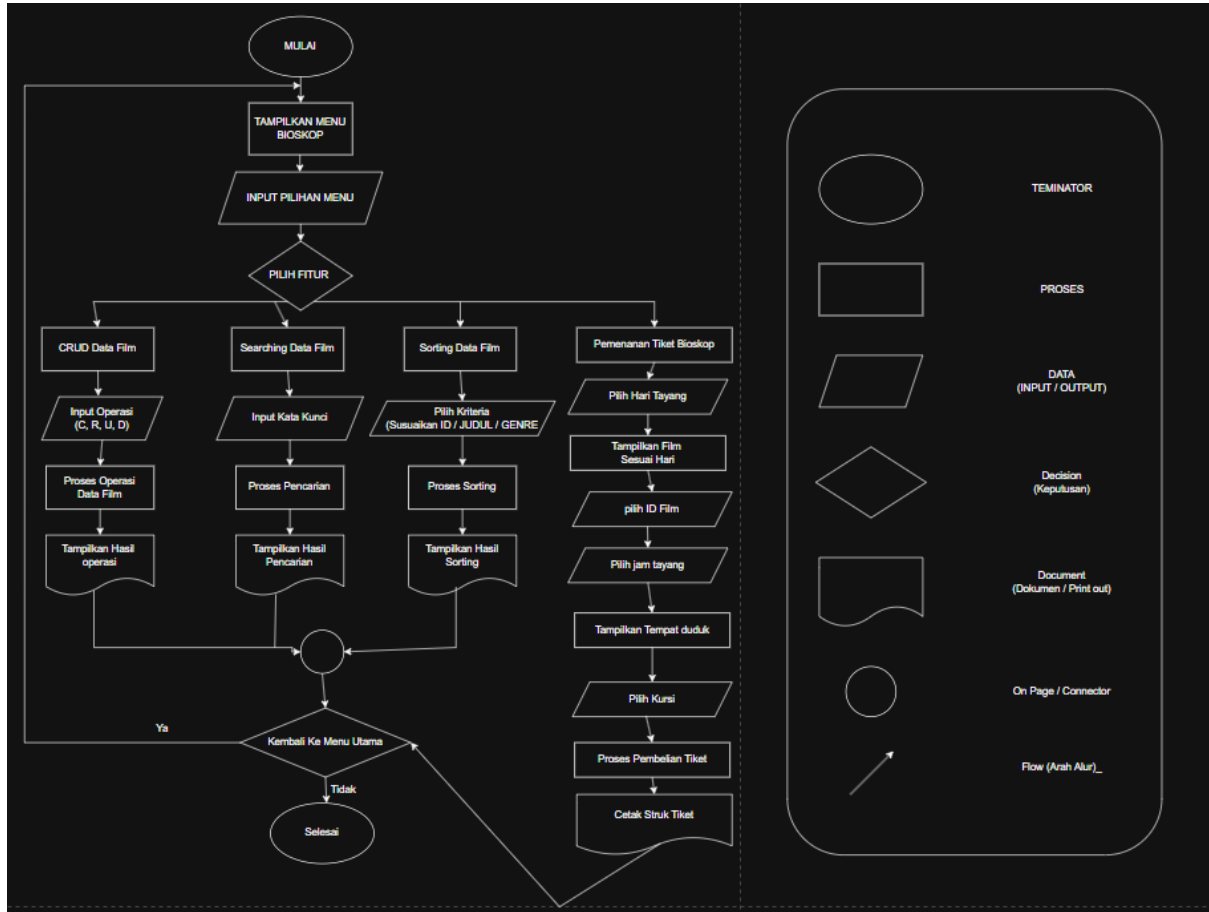
Program hanya menggunakan satu variabel sementara untuk proses pertukaran data:

```
Tiket temp;
```

Karena hanya menggunakan variabel tambahan berukuran tetap, maka kompleksitas ruang tambahannya adalah $O(1)$.

BAB V

PENGUJIAN



Gambar 6.1 menunjukkan flowchart alur penggunaan sistem pemesanan tiket bioskop, mulai dari memilih menu pada sistem, melakukan proses pemesanan tiket, hingga transaksi selesai dilakukan.

```

===== MENU BIOSKOP =====
1. Create
2. Read
3. Update
4. Delete
5. Cari Nama
6. Cari Kategori
7. Sort ID
8. Cari ID
9. Sort Nama
10. Beli Tiket
11. Sort Kursi Terbanyak
0. Keluar
  
```

Gambar 6.2 output awal

Pada gambar 6.2, Itu Adalah ouput atau tampilan awal pada sistem yang kami buat, kita memiliki kurang lebih 11 – 12 fitur yang tersedia, kurang lebih nya seperti CRUD, Sort , Search dan fitur tambahan lain nya.

Untuk mulai memesan tiket, hal pertama yang di lakukan Adalah mengecek atau ketahui terlebih dahulu ketersediaan film yang kita ingin tonton itu ada atau tidak, dengan cara:

Pilih: 2

ID	FILM	KATEGORI	KURSI	JAM	HARI	STATUS
1	Avengers	Action	20	10:00, 14:00, 19:00	Senin	Tersedia
2	Batman	Action	10	11:00, 15:00, 20:00	Selasa	Tersedia
3	Spiderman	Action	5	09:00, 13:00, 18:00	Rabu	Tersedia
4	Frozen	Animation	2	12:00, 16:00, 21:00	Kamis	Tersedia
5	Joker	Drama	0	10:00, 14:00, 19:00	Jumat	Habis
6	Titanic	Drama	25	11:00, 15:00, 20:00	Sabtu	Tersedia
7	Avatar	Sci-Fi	25	09:00, 13:00, 18:00	Minggu	Tersedia
8	Inception	Sci-Fi	25	12:00, 16:00, 21:00	Senin	Tersedia
9	Up	Animation	25	10:00, 14:00, 19:00	Selasa	Tersedia
10	Coco	Animation	25	11:00, 15:00, 20:00	Rabu	Tersedia
11	Interstellar	Sci-Fi	25	09:00, 13:00, 18:00	Kamis	Tersedia
12	Dune	Sci-Fi	25	12:00, 16:00, 21:00	Jumat	Tersedia
13	Cars	Animation	25	10:00, 14:00, 19:00	Sabtu	Tersedia
14	Minions	Animation	25	11:00, 15:00, 20:00	Minggu	Tersedia
15	Aladdin	Animation	25	09:00, 13:00, 18:00	Senin	Tersedia
16	Matrix	Sci-Fi	25	12:00, 16:00, 21:00	Selasa	Tersedia
17	Thor	Action	25	10:00, 14:00, 19:00	Rabu	Tersedia
18	Iron Man	Action	25	11:00, 15:00, 20:00	Kamis	Tersedia
19	Hulk	Action	25	09:00, 13:00, 18:00	Jumat	Tersedia
20	Doctor Strange	Sci-Fi	25	12:00, 16:00, 21:00	Sabtu	Tersedia
21	Transformers	Sci-Fi	25	10:00, 14:00, 19:00	Minggu	Tersedia
22	Kungfu Panda	Animation	25	11:00, 15:00, 20:00	Senin	Tersedia

Gambar 6.3 fitur READ

Pada gambar tersebut kita memilih fitur 2 (READ) untuk menunjukan atau membaca semua film yang tersedia, fitur ini menjelaskan semua nya, di mulai dari jam tayang, hari tayang , dan ketersediaan kursi ada atau tidak nya.

Lalu setelah mengecek ketersediaan film nya, kita lanjut ke fitur 10 (BELI TIKET)



Gambar 6.4 fitur BELI TIKET

Output yang muncul jika kita membeli tiket Adalah pilihan hari (kita mau nonton hari apa).
Lalu setelah kita memilih hari, output selanjutnya Adalah seperti berikut:

```

===== FILM HARI SELASA =====
ID  FILM          JAM              KURSI
2   Batman        11:00, 15:00, 20:00  10
9   Up            10:00, 14:00, 19:00  25
16  Matrix        12:00, 16:00, 21:00  25
23  Moana         09:00, 13:00, 18:00  25
30  Fast Furious  11:00, 15:00, 20:00  25

Masukkan ID Film: 2

```

Gambar 6.5 output lanjutan dari BELI TIKET

Akan terlihat film – film yang tersedia pada hari yang kita pilih (saya memilih hari selasa, maka yang muncul film Batman, Up, dan lain lain)

```

Masukkan ID Film: 2

Film : Batman
Hari : Selasa
Kursi tersedia : 10

=== PILIH JAM ===
1. 11:00
2. 15:00
3. 20:00
Pilih jam: 1

Jam dipilih : 11:00

=== TEMPAT DUDUK ===

A1 [X]  A2 [X]  A3 [X]  A4 [X]  A5 [X]
B1 [X]  B2 [X]  B3 [X]  B4 [X]  B5 [X]
C1 [X]  C2 [X]  C3 [X]  C4 [X]  C5 [X]
D1 [ ]  D2 [ ]  D3 [ ]  D4 [ ]  D5 [ ]
E1 [ ]  E2 [ ]  E3 [ ]  E4 [ ]  E5 [ ]

[ ] = tersedia
[X] = sudah dipesan

```

Gambar 6.6 terlihat proses lanjutan pada fitur beli tiket.

Setelah pengguna memasukkan ID film yang dipilih, sistem akan menampilkan informasi film, jam tayang yang tersedia, serta tampilan tempat duduk bioskop. Pada tampilan tersebut, tanda [] menunjukkan kursi yang masih tersedia. Sedangkan tanda [X] menunjukkan kursi yang sudah dipesan oleh pengguna lain.

```
Jumlah tiket: 5
Pilih kursi: e1
Pilih kursi: e2
Pilih kursi: e3
Pilih kursi: e4
Pilih kursi: e5

Pembelian berhasil!
```

Gambar 6.7

Pengguna dapat memilih beberapa kursi sesuai jumlah tiket yang dibeli. Setelah kursi berhasil dipilih, sistem akan memproses transaksi dan menampilkan pesan bahwa pembelian tiket berhasil dilakukan.

```
=====
                        CINEMA XXI
                        ENJOY THE MOVIE!
=====
ID Transaksi : TX1779270839351
Tanggal      : 23 Mei 2026
Waktu        : 12:30
-----
Film          : Batman
Kategori      : Action
Hari Tayang   : Selasa
Jam Tayang    : 11:00
Studio        : Studio 1
-----
Kursi         : E1, E2, E3, E4, E5
Jumlah Tiket  : 5
Harga/Tiket   : Rp50000
-----
TOTAL BAYAR   : Rp250000
=====
                TERIMA KASIH TELAH MENONTON
                MOHON DATANG KEMBALI :)
=====
```

Gambar 6.8 output terakhir yaitu STRUK PEMESANAN

Struk tersebut menampilkan informasi transaksi seperti ID transaksi, tanggal, waktu, nama film, kategori film, hari tayang, jam tayang, studio, kursi yang dipilih, jumlah tiket, harga tiket, serta total pembayaran. Output ini dibuat untuk mensimulasikan tampilan struk pembelian tiket bioskop seperti pada sistem bioskop asli.

BAB VI

KESIMPULAN

Berdasarkan hasil perancangan dan implementasi program, dapat disimpulkan bahwa sistem pemesanan tiket bioskop berbasis Java ini berhasil dibuat dan berjalan dengan baik sesuai tujuan penelitian. Program mampu mengelola data film menggunakan fitur CRUD, melakukan proses searching dan sorting data, serta menjalankan fitur pemesanan tiket lengkap dengan pemilihan kursi dan pencetakan struk pembelian.

Selain itu, project ini juga berhasil mengimplementasikan konsep Struktur Data dan Algoritma seperti array object, array dua dimensi, Linear Search, Binary Search, Bubble Sort, dan Selection Sort dalam kasus nyata secara sederhana. Dengan adanya project ini, penulis dapat lebih memahami penerapan algoritma dan struktur data dalam pengembangan sistem berbasis Java console.

Meskipun sistem masih sederhana dan belum menggunakan database maupun GUI, program ini sudah mampu mensimulasikan proses pemesanan tiket bioskop dengan cukup baik dan dapat dikembangkan lebih lanjut menjadi sistem yang lebih modern di masa mendatang.